

Assignment 1: Basic operations in Python

1. Read two integers and print sum, difference, product, and integer division.

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print("Sum:", a + b)
print("Difference:", a - b)
print("Product:", a * b)
print("Integer Division:", a // b)
```

```
Enter first number: 12
Enter second number: 12
Sum: 24
Difference: 0
Product: 144
Integer Division: 1
```

2. Write a function `is_even(n)` that returns True if `n` is even, else False.

```
def is_even(n):
    return n % 2 == 0

print(is_even(4))
print(is_even(7))
```

```
True
False
```

3. From a list of integers, create a new list with only even numbers.

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]

even_nums = [x for x in nums if x % 2 == 0]

print("Even numbers:", even_nums)
```

```
Even numbers: [2, 4, 6, 8]
```

4. Write a Python program `student_marks_analyzer.py` that asks the user to enter the number of students in a class. Then ask the user to enter the marks (0–100) for each student, one by one, and store all the marks so that they can be processed later. After all marks are entered, the program should display:

- The list of all marks entered.
- The total of all marks.
- The average marks of the class.
- The highest and lowest marks in the class.
- How many students passed (marks ≥ 40).
- How many students failed (marks < 40).
- Display the marks in reverse order of entry.
- Display only those marks that are even numbers.

```
n = int(input("Enter number of students: "))

marks = []

for i in range(n):
    m = int(input(f"Enter marks for student {i+1}: "))
    marks.append(m)

print("Marks:", marks)

total = sum(marks)
average = total / n

print("Total:", total)
print("Average:", average)

print("Highest:", max(marks))
print("Lowest:", min(marks))

passed = len([m for m in marks if m >= 40])
failed = len([m for m in marks if m < 40])

print("Passed:", passed)
print("Failed:", failed)

print("Marks in reverse:", marks[::-1])

even_marks = [m for m in marks if m % 2 == 0]
print("Even marks:", even_marks)
```

```
Enter number of students: 4
Enter marks for student 1: 20
Enter marks for student 2: 50
Enter marks for student 3: 60
Enter marks for student 4: 70
Marks: [20, 50, 60, 70]
Total: 200
Average: 50.0
Highest: 70
Lowest: 20
Passed: 3
Failed: 1
Marks in reverse: [70, 60, 50, 20]
Even marks: [20, 50, 60, 70]
```

Assignment 2

Task 1: Customer Purchase Decision

Each customer entering the grocery store may or may not buy at least one item. Historical data shows that 30% of customers make a purchase. You observe 1000 independent customers.

- Define a random variable representing the outcome of a customer visit
- Identify all possible values of the random variable
- Decide the appropriate probability model
- Simulate the experiment using Python
- Compute the mean and variance
- Visualize the outcomes

```
import numpy as np
import matplotlib.pyplot as plt

n_customer = 1000
p_purchase = 0.3

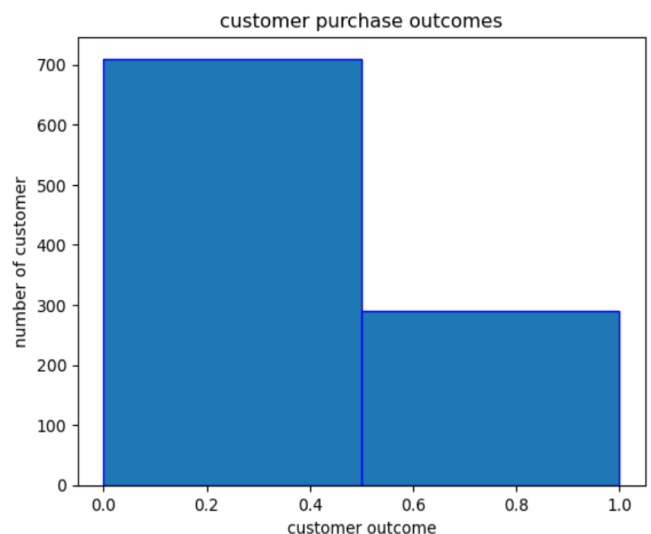
data = np.random.binomial(1, p_purchase, n_customer)

total_purchase = np.sum(data)
mean_value = np.mean(data)
variance_value = np.var(data)

print("total customer : ", n_customer)
print("total purchase : ", total_purchase)
print("mean : ", mean_value)
print("variance : ", variance_value)

plt.figure()
plt.hist(data, bins=2, edgecolor = 'blue')
plt.xlabel('customer outcome')
plt.ylabel('number of customer')
plt.title('customer purchase outcomes')
plt.show()
```

```
total customer : 1000
total purchase : 290
mean : 0.29
variance : 0.20589999999999997
```



Task 2: Number of Purchasing Customers in a Fixed Group

During a discount hour, a fixed number of customers enter the store. Exactly 50 customers enter the store in one hour. Each customer independently decides whether to purchase. The probability of purchase remains constant.

- Define a random variable representing the number of customers who make purchases
- Identify the structure of the experiment (fixed trials, success/failure)
- Select a suitable probability distribution
- Compute and plot the probability mass function (PMF)
- Interpret the most likely outcome

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom

n = 50
p = 0.3

x = np.arange(0, n+1)

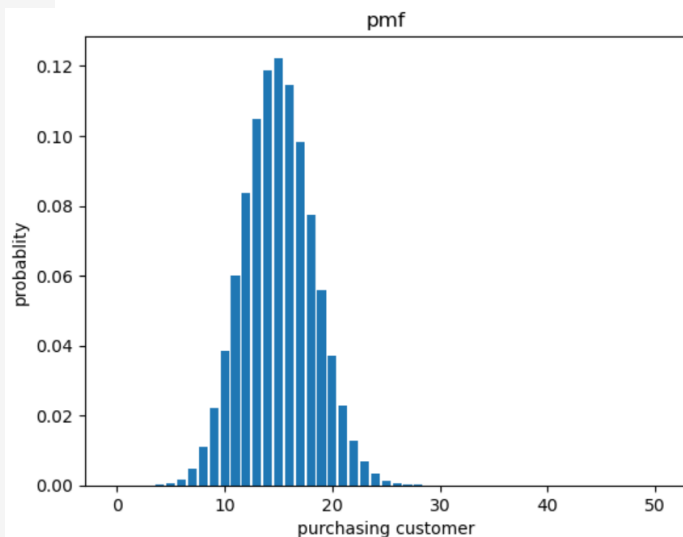
pmf = binom.pmf(x, n, p)

mean = n * p
variance = n * p * (1 - p)
mode = int(np.floor((n + 1) * p))

print("mean:", mean)
print("variance:", variance)
print("mode:", mode)

plt.figure()
plt.bar(x, pmf)
plt.xlabel('purchasing customer')
plt.ylabel('probability')
plt.title('pmf')
plt.show()
```

```
mean: 15.0
variance: 10.5
mode: 15
```



Task 3: Checkout Demand Over Time

Customers arrive at checkout counters continuously. The average number of customers checking out per hour is 10. Checkout events occur randomly and independently. Exact arrival times are unpredictable.

- Define a random variable representing the number of checkouts in one hour
- Identify the characteristics of the arrival process
- Select a probability distribution suitable for event counts over time
- Compute:
 - Probability of exactly 8 checkouts in one hour
 - Probability of more than 15 checkouts in one hour
- Visualize the distribution and explain how it helps staffing decisions

```
import math
from scipy.stats import poisson
import matplotlib.pyplot as plt

lam = 10

def poisson_pmf(k, lam):
    return (math.exp(-lam) * lam**k) / math.factorial(k)

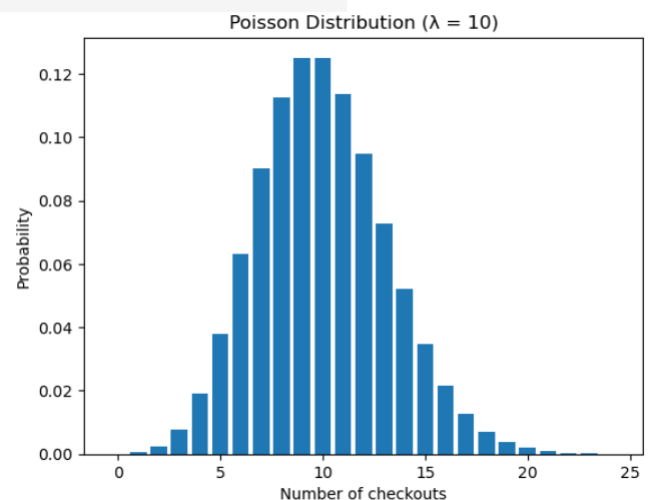
p_exact_8 = poisson_pmf(8, lam)

p_more_15 = 1 - poisson.cdf(15, lam)

print("P(X = 8):", p_exact_8)
print("P(X > 15):", p_more_15)

x = list(range(0, 25))
y = [poisson_pmf(k, lam) for k in x]

plt.bar(x, y)
plt.xlabel("Number of checkouts")
plt.ylabel("Probability")
plt.title("Poisson Distribution ( $\lambda = 10$ )")
plt.show()
```



P(X = 8): 0.11259903214901998

P(X > 15): 0.04874040330397866

Task 4: First Successful Promotion Response

A store promotion is announced repeatedly to customers until someone responds. Each promotional interaction has a 25% chance of success. Customer responses are independent.

- Define a random variable representing the number of interactions until the first positive response
- Identify whether past interactions affect future outcomes
- Choose a suitable probability distribution
- Compute the probability that the first success occurs on the 4th interaction
- Plot the PMF for the first 10 interactions

```
import matplotlib.pyplot as plt

p = 0.25

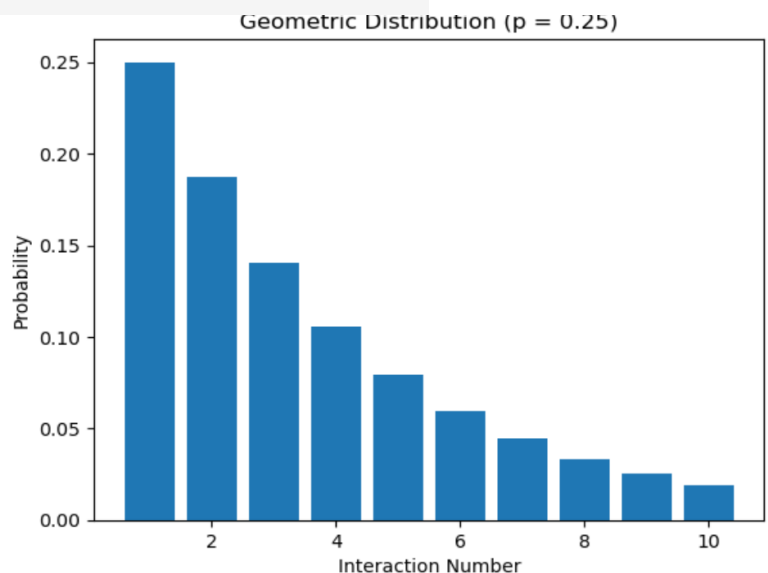
def geometric_pmf(k, p):
    return (1 - p)**(k - 1) * p

p_k4 = geometric_pmf(4, p)
print("P(X = 4):", p_k4)

x = list(range(1, 11))
y = [geometric_pmf(k, p) for k in x]

plt.bar(x, y)
plt.xlabel("Interaction Number")
plt.ylabel("Probability")
plt.title("Geometric Distribution (p = 0.25)")
plt.show()
```

P(X = 4): 0.10546875



Task 5: Quality Control – Defective Grocery Items

At the end of each day, packaged food items are inspected. Each item is either defective or non-defective. The probability of defect is small. A large number of items are handled daily.

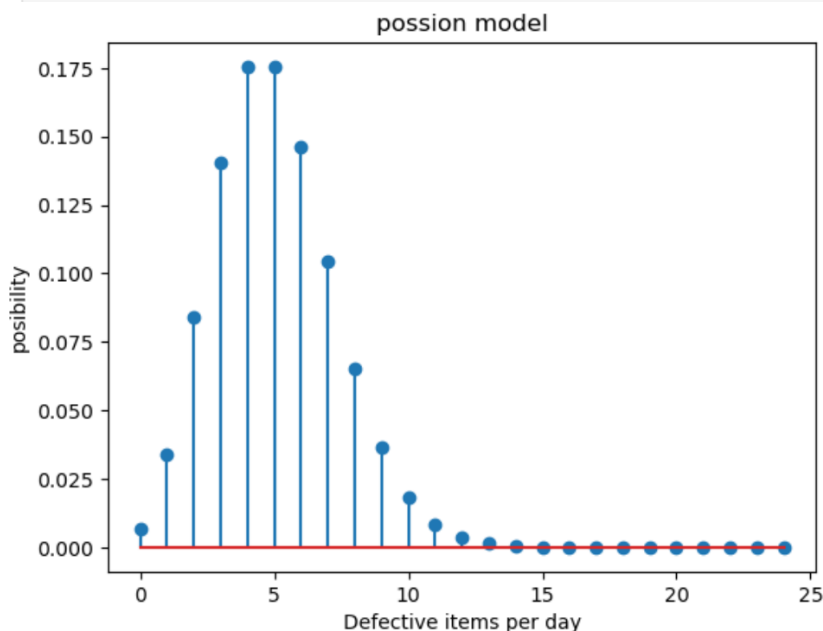
- Define a suitable random variable for defects
- Decide whether to model defects individually or as counts
- Select an appropriate probability distribution
- Justify whether approximation methods can be applied
- Interpret results in terms of quality and waste reduction

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import poisson

n = 5000
p = 0.001
lam= n*p

k= np.arange(0,25)
pmf = poisson.pmf(k , lam)

plt.figure()
plt.stem(k,pmf)
plt.xlabel("Defective items per day")
plt.ylabel("possibility")
plt.title("poission model")
plt.show()
```



Assignment 2A

1 Module 1: The Real Estate Data Pipeline

Goal: Practice NumPy for synthesis, Pandas for cleaning, and Seaborn for correlation analysis.

1.1 Task

1.1: Data Synthesis

Generate a synthetic dataset of 100 houses:

- Create a 1D NumPy array X (Square Footage) with 100 random integers [800, 5000].
- Create a target array y (Price) using $y = 150X + \epsilon$, where ϵ is Gaussian noise via `np.random.normal(0, 50000, 100)`.

1.2 Task

1.2: Wrangling & Encoding

- Convert arrays to a DataFrame. Introduce 5 NaN values in Price and fill them with the median.
- Add a categorical column Location ('Urban', 'Suburban', 'Rural') and apply One-Hot Encoding.

1.3 Task

1.3: Visualizing Relationships

- Plot a Seaborn Heatmap of the correlation matrix.
- Use `plt.subplots` to show a Histogram of prices and a Scatter Plot of SqF t vs P rice.


```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42)

X = np.random.randint(800, 5001, 100)
epsilon = np.random.normal(0, 50000, 100)
y = 150 * X + epsilon

df = pd.DataFrame({
    "SqFt": X,
    "Price": y
})

nan_indices = np.random.choice(df.index, 5, replace=False)
df.loc[nan_indices, "Price"] = np.nan

df["Price"].fillna(df["Price"].median(), inplace=True)

locations = np.random.choice(["Urban", "Suburban", "Rural"], 100)
df["Location"] = locations

df = pd.get_dummies(df, columns=["Location"])

corr = df.corr()

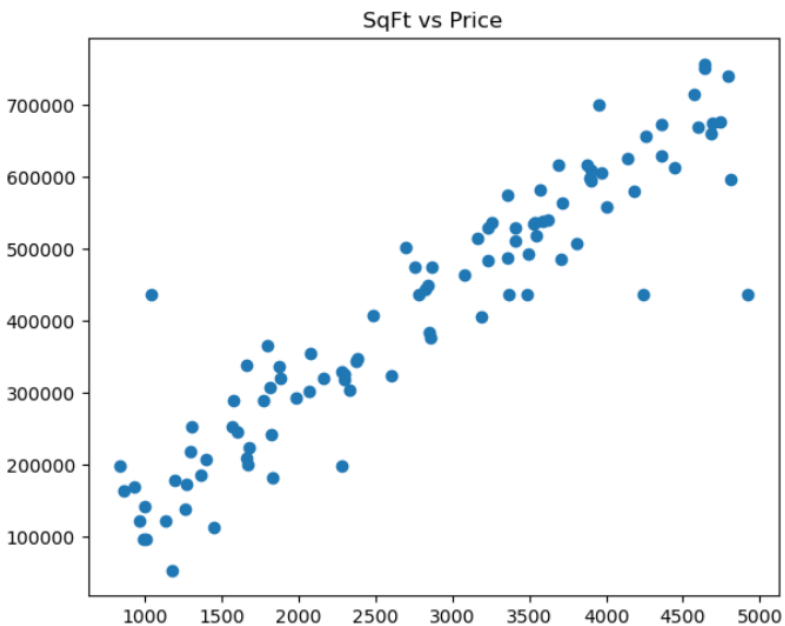
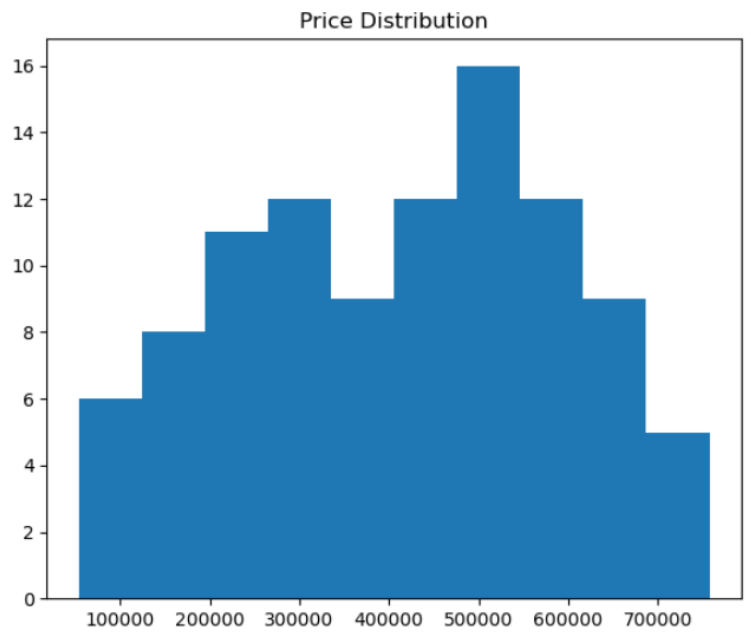
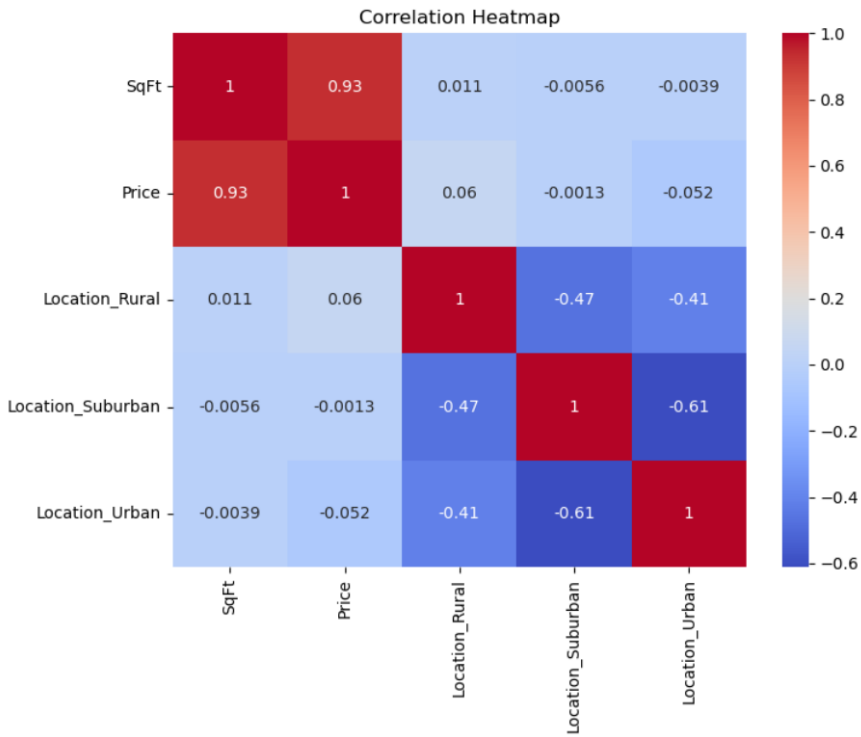
plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")

fig, axes = plt.subplots(1, 2, figsize=(12,5))

axes[0].hist(df["Price"])
axes[0].set_title("Price Distribution")

axes[1].scatter(df["SqFt"], df["Price"])
axes[1].set_title("SqFt vs Price")

plt.tight_layout()
plt.show()
```



2 Module 2: Probability & Store Analytics

Goal: Implement discrete probability distributions using `scipy.stats`.

2.1 Task

2.1: Bernoulli Trials

30% of 1,000 customers make a purchase.

- Simulate 1,000 trials. Compute empirical mean ($\bar{x}$) and variance (s^2).
- Compare these to the theoretical values $\mu = p$ and $\sigma^2 = p(1 - p)$.

2.2 Task

2.2: Binomial Distribution

Exactly 50 customers enter during a discount hour ($p = 0.3$).

- Plot the Probability Mass Function (PMF) for all possible outcomes (0–50).
- Identify the most likely number of customers to make a purchase.

2.3 Task

2.3: Poisson Process

Average checkout rate is $\lambda = 10$ per hour.

- Calculate $P(X = 8)$ and $P(X > 15)$.
- Visualize the distribution. Explain how this helps in staffing decisions.

2.4 Task

2.4: Geometric Waiting Times

A promotion has a 25% success rate.

- Calculate the probability the first success occurs on the 4th interaction.
- Plot the PMF for the first 10 interactions.

2.5 Task

2.5: Quality Control (Poisson Approximation)

In a batch of 5,000 items with a very small defect rate ($p = 0.001$):

- Justify the use of Poisson as an approximation for Binomial in this context.
- Compute the probability of finding more than 10 defects.

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import bernoulli, binom, poisson, geom

np.random.seed(42)

p = 0.3
trials = bernoulli.rvs(p, size=1000)

emp_mean = np.mean(trials)
emp_var = np.var(trials)

theo_mean = p
theo_var = p * (1 - p)

print("Empirical Mean:", emp_mean)
print("Empirical Variance:", emp_var)
print("Theoretical Mean:", theo_mean)
print("Theoretical Variance:", theo_var)

n = 50
x = np.arange(0, n + 1)
pmf_binom = binom.pmf(x, n, p)

plt.figure()
plt.bar(x, pmf_binom)
plt.title("Binomial PMF (n=50, p=0.3)")
plt.xlabel("Number of Purchases")
plt.ylabel("Probability")

most_likely = x[np.argmax(pmf_binom)]
print("Most Likely Purchases:", most_likely)

lam = 10
p_x8 = poisson.pmf(8, lam)
p_gt15 = 1 - poisson.cdf(15, lam)

print("P(X=8):", p_x8)
print("P(X>15):", p_gt15)

x_pois = np.arange(0, 25)
pmf_pois = poisson.pmf(x_pois, lam)

```

```

x_pois = np.arange(0, 25)
pmf_pois = poisson.pmf(x_pois, lam)

plt.figure()
plt.bar(x_pois, pmf_pois)
plt.title("Poisson PMF ( $\lambda=10$ )")
plt.xlabel("Checkouts")
plt.ylabel("Probability")

p_geom = 0.25
p_k4 = geom.pmf(4, p_geom)
print("P(X=4):", p_k4)

x_geom = np.arange(1, 11)
pmf_geom = geom.pmf(x_geom, p_geom)

plt.figure()
plt.bar(x_geom, pmf_geom)
plt.title("Geometric PMF (p=0.25)")
plt.xlabel("Trial")
plt.ylabel("Probability")

n_large = 5000
p_small = 0.001
lam_approx = n_large * p_small

p_more_10 = 1 - poisson.cdf(10, lam_approx)
print("P(X>10) using Poisson Approx:", p_more_10)

plt.show()

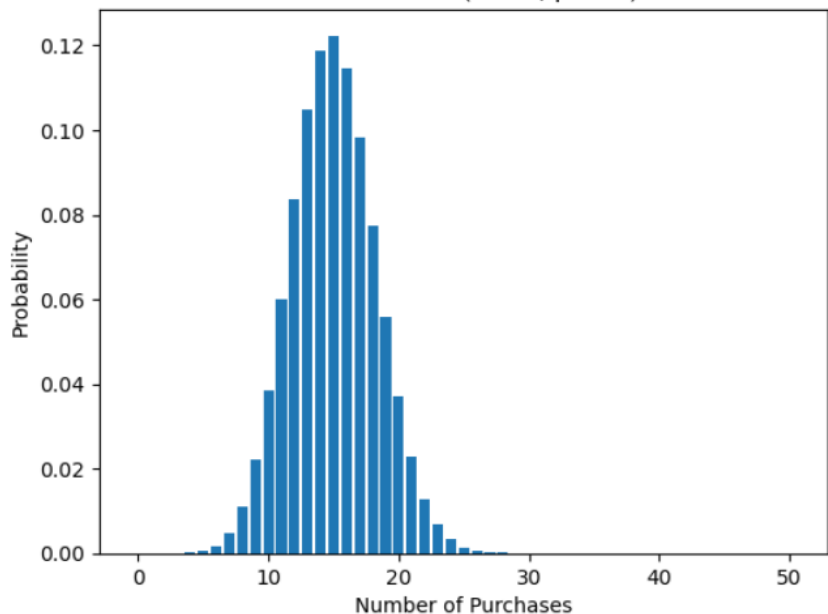
```

```

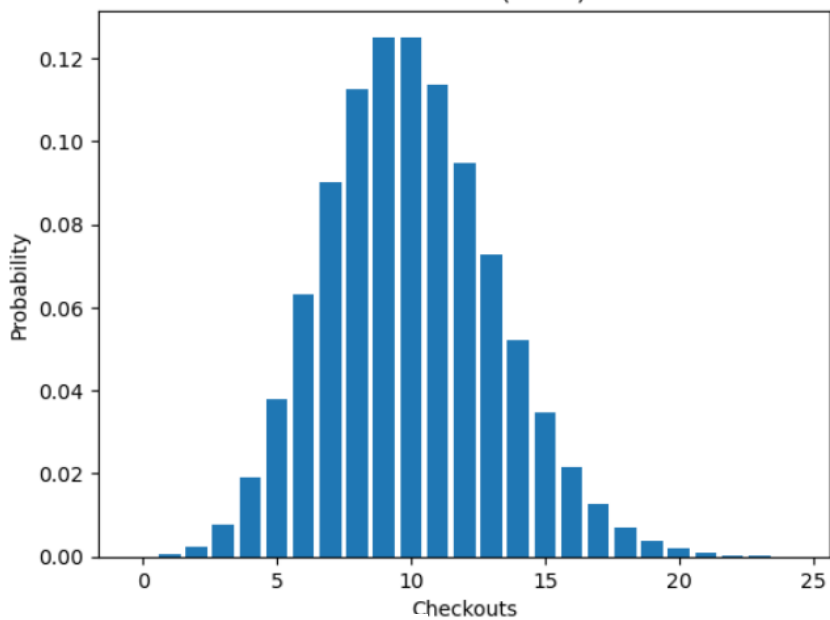
Empirical Mean: 0.288
Empirical Variance: 0.20505599999999993
Theoretical Mean: 0.3
Theoretical Variance: 0.21
Most Likely Purchases: 15
P(X=8): 0.11259903214902009
P(X>15): 0.04874040330397866
P(X=4): 0.10546875
P(X>10) using Poisson Approx: 0.013695268598382881

```

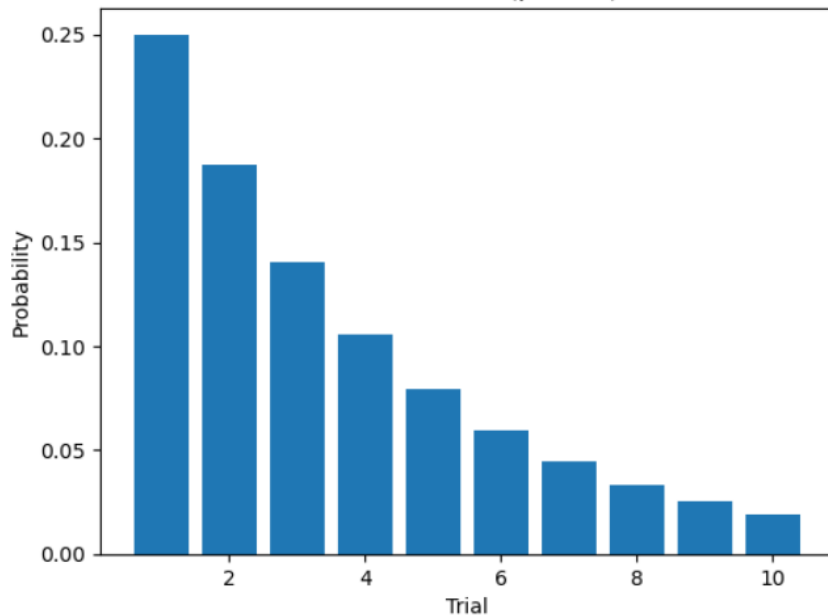
Binomial PMF ($n=50, p=0.3$)



Poisson PMF ($\lambda=10$)



Geometric PMF ($p=0.25$)



Assignment 3

Find the Principal Components. How many components would you use for any analysis or reconstruction?

```
from ucimlrepo import fetch_ucirepo

# fetch dataset
wine = fetch_ucirepo(id=109)

X = wine.data.features
y = wine.data.targets

print(wine.metadata)
print(wine.variables) |
```

	name	role	type	demographic
0	class	Target	Categorical	None
1	Alcohol	Feature	Continuous	None
2	Malicacid	Feature	Continuous	None
3	Ash	Feature	Continuous	None
4	Alcalinity_of_ash	Feature	Continuous	None
5	Magnesium	Feature	Integer	None
6	Total_phenols	Feature	Continuous	None
7	Flavanoids	Feature	Continuous	None
8	Nonflavanoid_phenols	Feature	Continuous	None
9	Proanthocyanins	Feature	Continuous	None
10	Color_intensity	Feature	Continuous	None
11	Hue	Feature	Continuous	None
12	OD280_OD315_of_diluted_wines	Feature	Continuous	None
13	Proline	Feature	Integer	None

	description	units	missing_values
0	None	None	no
1	None	None	no
2	None	None	no
3	None	None	no
4	None	None	no
5	None	None	no
6	None	None	no
7	None	None	no
8	None	None	no
9	None	None	no
10	None	None	no
11	None	None	no
12	None	None	no
13	None	None	no

```

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np

X_scaled = StandardScaler().fit_transform(X)

pca = PCA()
X_pca = pca.fit_transform(X_scaled)

explained_variance = pca.explained_variance_ratio_
cumulative_variance = np.cumsum(explained_variance)

print("Explained Variance Ratio:", explained_variance)
print("Cumulative Variance:", cumulative_variance)

plt.plot(range(1, len(cumulative_variance)+1), cumulative_variance, marker='o')
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Explained Variance")
plt.show()

n_components = np.argmax(cumulative_variance >= 0.95) + 1
print("Optimal Number of Components:", n_components)

pca_final = PCA(n_components=n_components)
X_reduced = pca_final.fit_transform(X_scaled)

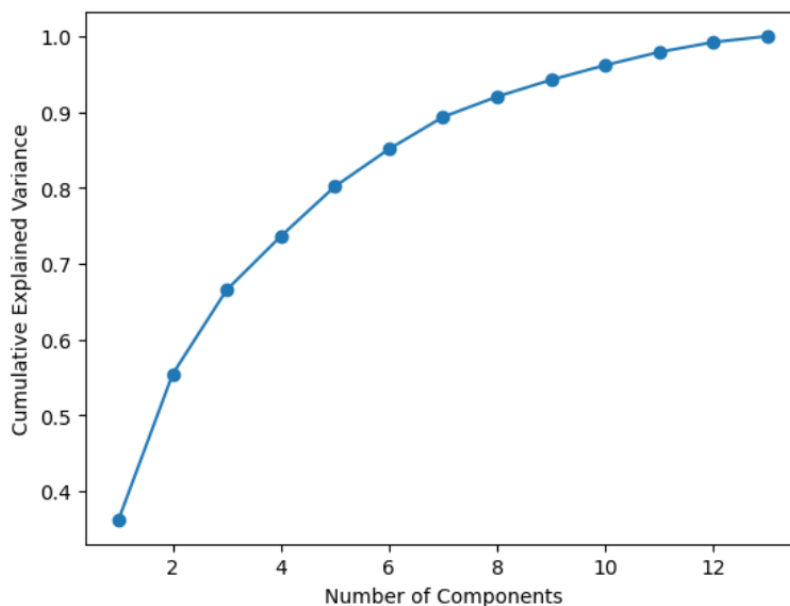
print("Reduced Data Shape:", X_reduced.shape)

```

```

Explained Variance Ratio: [0.36198848 0.1920749  0.11123631 0.0706903  0.06563294 0.04935823
 0.04238679 0.02680749 0.02222153 0.01930019 0.01736836 0.01298233
 0.00795215]
Cumulative Variance: [0.36198848 0.55406338 0.66529969 0.73598999 0.80162293 0.85098116
 0.89336795 0.92017544 0.94239698 0.96169717 0.97906553 0.99204785
 1.          ]

```



Optimal Number of Components: 10
Reduced Data Shape: (178, 10)

Assignment 4

Experiment 1: The Law of Large Numbers (The Running Average)

The LLN states that as the number of trials n increases, the sample mean $\bar{X}_n$ converges to the theoretical expected value μ .

Task

- Simulate $n = 10,000$ Bernoulli trials (coin flips) with $p = 0.7$.
- Calculate the *cumulative* average: $A_n = \frac{1}{n} \sum_{i=1}^n X_i$.
- Plot A_n against n . Add a horizontal dashed line at $y = 0.7$.

```
import numpy as np
import matplotlib.pyplot as plt

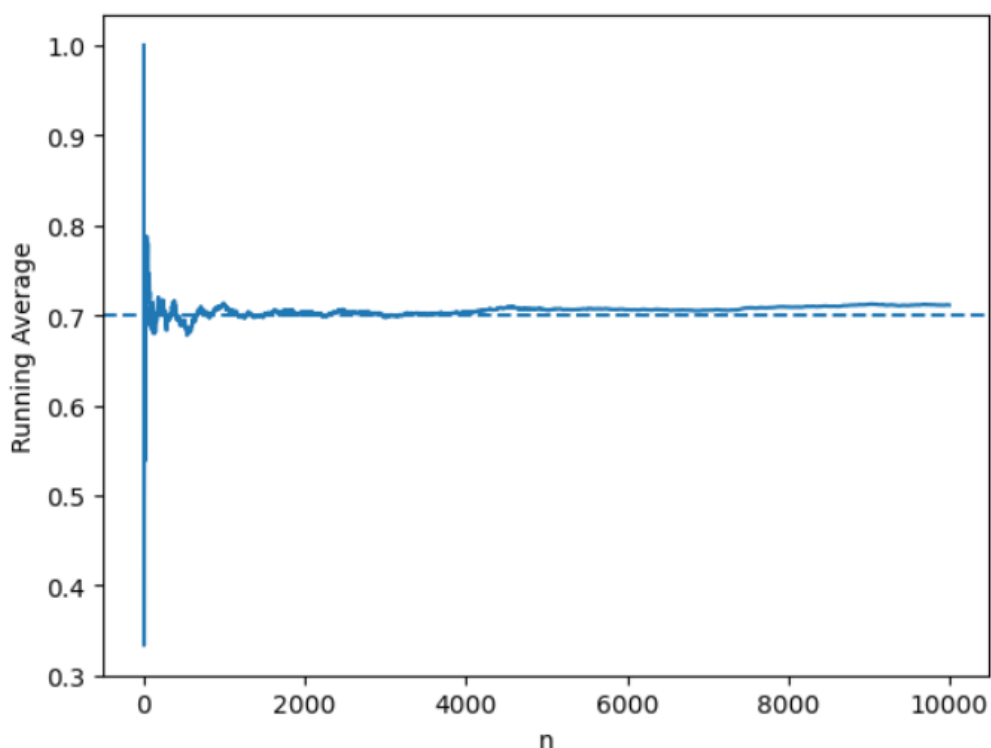
np.random.seed(42)

n = 10000
p = 0.7

trials = np.random.binomial(1, p, n)

cumulative_avg = np.cumsum(trials) / np.arange(1, n+1)

plt.plot(cumulative_avg)
plt.axhline(y=0.7, linestyle='--')
plt.xlabel("n")
plt.ylabel("Running Average")
plt.show()
```



Experiment 2: The Central Limit Theorem (Sum of Dice)

The CLT states that the sum (or mean) of n independent and identically distributed (i.i.d.) random variables tends toward a normal distribution, regardless of the original distribution shape.

Task

1. Generate $m = 1,000$ samples.
2. Each sample should consist of the mean of k fair 6-sided dice.
3. Repeat this for $k \in \{1, 2, 5, 30\}$.
4. Plot four histograms side-by-side to show the evolution of the distribution.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(42)

m = 1000
k_values = [1, 2, 5, 30]

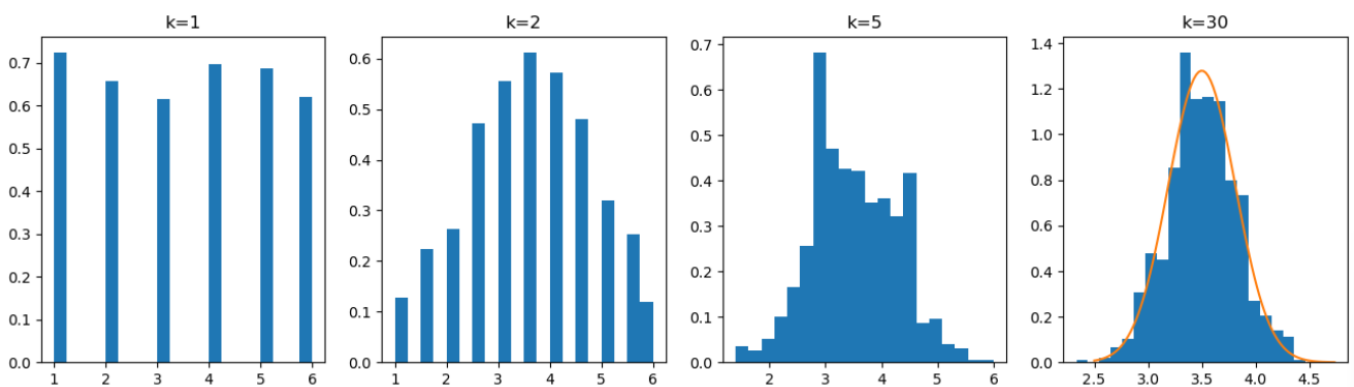
fig, axes = plt.subplots(1, 4, figsize=(16,4))

for i, k in enumerate(k_values):
    samples = np.mean(np.random.randint(1, 7, (m, k)), axis=1)
    axes[i].hist(samples, bins=20, density=True)
    axes[i].set_title(f"k={k}")

k = 30
samples = np.mean(np.random.randint(1, 7, (m, k)), axis=1)
mu = 3.5
sigma = np.sqrt(35/12) / np.sqrt(k)

x = np.linspace(min(samples), max(samples), 100)
axes[-1].plot(x, norm.pdf(x, mu, sigma))

plt.show()
```



Experiment 3: The Skewness Challenge

To prove the CLT works for non-symmetric distributions, we will use the Exponential distribution, which is heavily right-skewed.

Task

- Let $X \sim \text{Exp}(\lambda = 0.5)$. Note that $\mu = 1/\lambda = 2$.
- Draw 500 samples of size $n = 50$.
- Plot a Q-Q (Quantile-Quantile) plot of the sample means against a theoretical normal distribution.

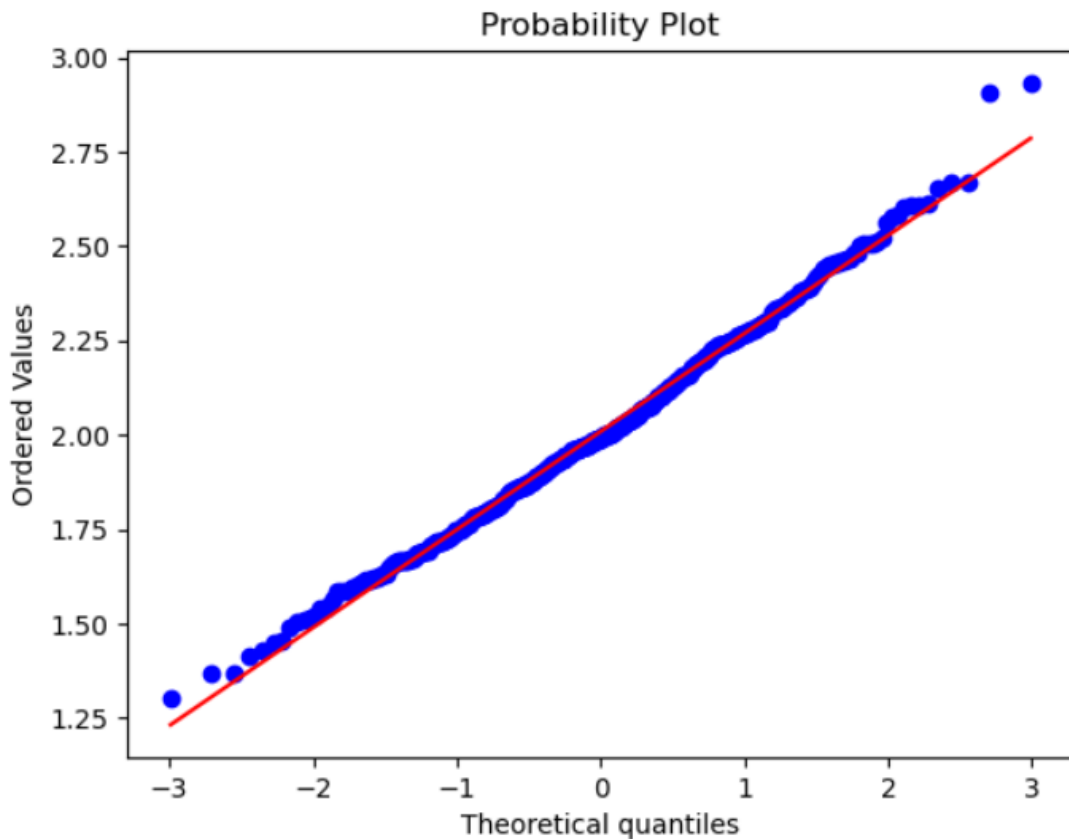
```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

np.random.seed(42)

lam = 0.5
n = 50
samples = 500

data = np.random.exponential(1/lam, (samples, n))
means = np.mean(data, axis=1)

stats.probplot(means, dist="norm", plot=plt)
plt.show()
```



Experiment 4: The Digital Galton Board

This experiment visualizes the Binomial approximation to the Normal distribution.

Task

- Simulate a ball falling through $N = 100$ levels of pegs.
- At each level, the ball moves right (+1) or left (0) with $p = 0.5$.
- The final position is $S = \sum_{i=1}^N X_i$.
- Run this for 5,000 balls and plot the resulting distribution.

[Image of Galton Board probability distribution]

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
N = 100
balls = 5000

moves = np.random.binomial(1, 0.5, (balls, N))
positions = np.sum(moves, axis=1)

plt.hist(positions, bins=30, density=True)
plt.xlabel("Final Position")
plt.ylabel("Probability Density")
plt.show()
```

